Lab3: Computer Architectuur

Stijn Verhaeghe & Andres Bosman

February 2026

1 Introduction

In dit labo onderzoeken we de invloed van een Warp. Een warp is een groep van 32 threads die volgens het SIMT-principe (Single Instruction, Multiple Threads) worden uitgevoerd. Threads binnen een block worden opgedeeld in warps; zodra een block bijvoorbeeld 33 threads bevat, worden er door de hardware 2 warps (totaal 64 thread-slots) toegewezen. Hierbij blijven 31 threads inactief, wat leidt tot inefficiënt gebruik van de beschikbare CUDA cores. Om dit te voorkomen, is het raadzaam om de block-grootte altijd een veelvoud van 32 te maken.

Wanneer de dataset groter is dan het totaal aantal beschikbare threads, gebruiken we Grid-Stride Loops. Hiermee kan elke thread via een 'stride' (stapgrootte) meerdere data-elementen verwerken, wat de schaalbaarheid en efficiëntie van de kernel bevordert.

In de tweede oefening van dit labo gaan we kijken naar het effect van thread divergence, als er een deel van de threads moelijkere operaties moeten uitvoeren dan gaat het deel dat eenvoudigere operaties moet uitvoeren altijd sneller klaar zijn en moeten wachten op de andere operaties. Als een warp bestaat uit trage en snelle threads dan zal deze warp pas klaar zijn als de traagste thread klaar is. Dit wil zeggen dat er een aantal cores niet benut kunnen worden.

2 oef1

De grafiek in Figuur 1 toont de gemiddelde uitvoeringstijd van de inversiekernel voor verschillende aantallen threads en blocks. De data laat zien dat de rekentijd afneemt naarmate het aantal threads per block toeneemt van 5 naar 1024 threads.

Een specifiek datapunt rond de warp-grootte (2^5 of 32 threads) is cruciaal: We dachten dat 32 threads beter presteren dan 31 of 33 threads. Dit bevestigt dat de GPU het efficiëntst werkt wanneer threads exact een volledige warp vullen. Bij 31 threads blijft er capaciteit onbenut, terwijl bij 33 threads de overhead van een tweede, bijna lege warp de gemiddelde tijd per pixel weer doet stijgen.

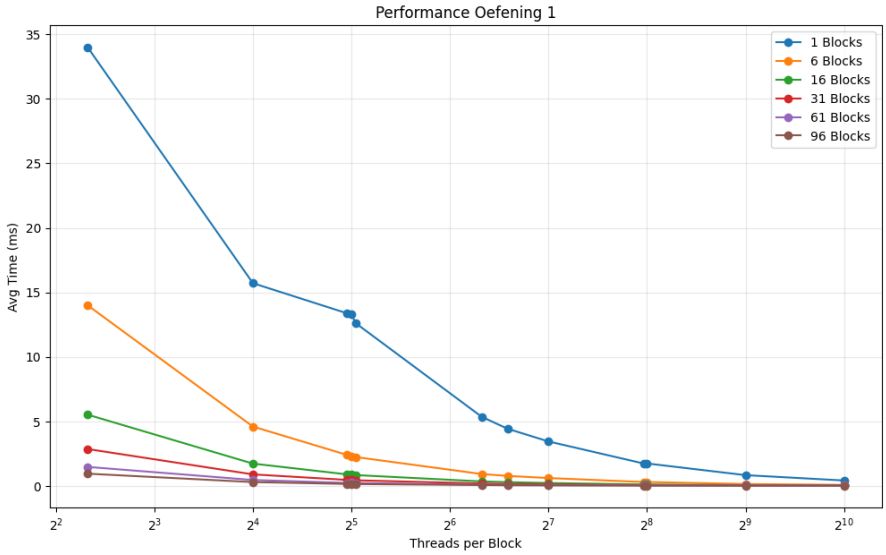


Figure 1: Resultaten oefening 1

3 oef 2

Voor de tweede oefening is het de bedoeling dat de rode pixels aangepast worden met een complexere formule ten opzichte van de blauwe of groene kleuren. Doordat het dataformaat als volgt eruit ziet 'RGBRGBRGB' wordt er in 1 Warp complexere berekeningen en eenvoudigere berekeningen uitgevoerd, hierdoor gaan een deel van de threads moeten wachten tot de threads met de complexere berekeningen klaar zijn. Hoe meer blokken gebruikt worden hoe rapper de berekeningen gedaan zijn.

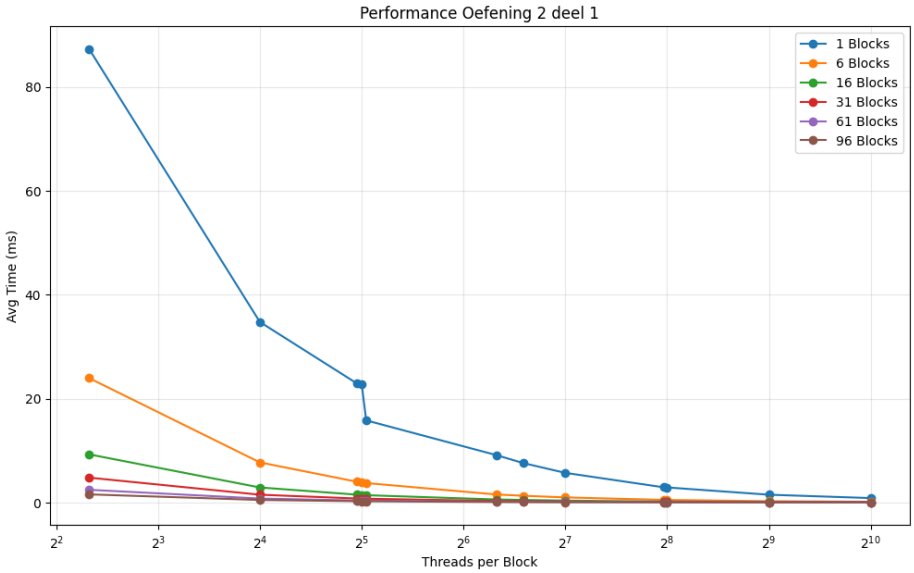


Figure 2: Resultaten oefening 2, deel 1

3.1 oefening 2 part 2

In het tweede deel van oefening 2 sorteren we de kleuren voordat we de kernel oproepen. In de vorige oefening werd elke warp opgehouden door de rode threads (ongeveer 1/3e van de warp) die de tragere Gaussian blur uitvoerden. Hierdoor waren de overige cores in de warp inactief terwijl ze wachtten op de trage berekening. Door te sorteren zitten vrijwel alle trage threads nu samen in dezelfde warps. Dit zorgt ervoor dat de trage berekeningen de snelle threads niet meer ophouden. Hierdoor kunnen meer trage threads tegelijkertijd berekend worden zonder dat er rekenkracht van snelle threads verloren gaat door onnodig wachten.

Uit figuur 3 kunnen we besluiten door de data eerst te sorteren, zien we dat de berekeningen al veel sneller zijn dan dat we de data niet sorteren (figuur 2). Dit hadden we ook verwacht de tragere berekeningen zitten nu te samen in 1 Warp.

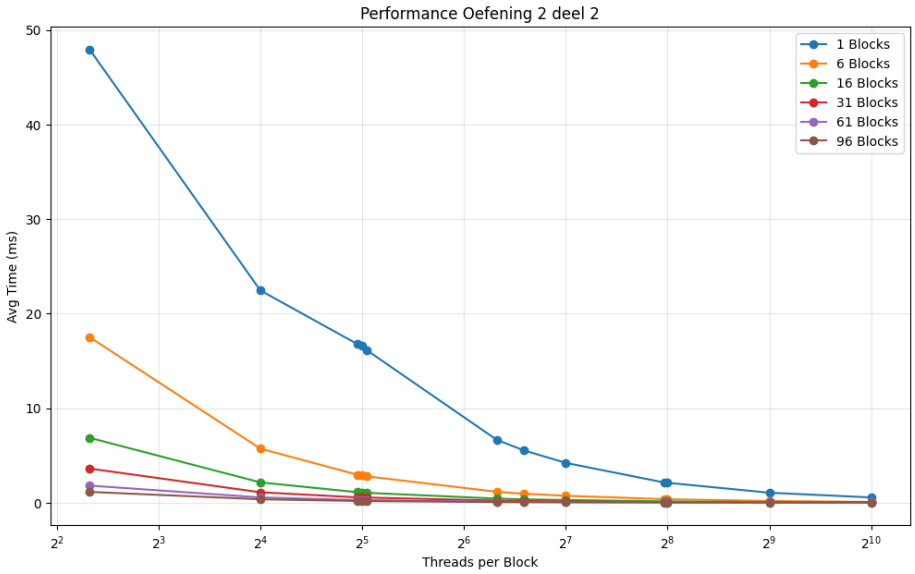


Figure 3: Resultaten oefening 2, deel 2

4 Appendix

4.1 Code voor oefening 1

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run 'convert <name>.<
    extension> <name>.ppm
 *
 */
#include <cstdint>      // Data types
#include <iostream>     // File operations

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
```

```
22      * Returns: Flattened image array.
23      *
24      * Noets:
25      *  - Images data is flattened per color, column, row.
26      *  - The first 3 data elements are the RGB components
27      *  - The first 3*M data elements represent the firts
            row of the image
28      *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,
            M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
29      *
30      */
31     // Try opening the file
32     FILE *imageFile;
33     imageFile=fopen("./anna.ppm","rb");
34     if(imageFile==NULL){
35         perror("ERROR: Cannot open output file");
36         exit(EXIT_FAILURE);
37     }
38
39     // Initialize empty image array
40     uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(
           uint8_t)+OFFSET);
41
42     // Read the image
43     fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t
           )+OFFSET, imageFile);
44
45     // Close the file
46     fclose(imageFile);
47
48     // Move the starting pointer and return the flattened
           image array
49     return image_array + OFFSET;
50 }
51
52 void save_image_array(uint8_t* image_array){
53     /*
54      * Save the data of an (RGB) image as a pixel map.
55      *
56      * Parameters:
57      *  - param1: The data of an (RGB) image as a 1D array
58      *
59      */
60     // Try opening the file
61     FILE *imageFile;
62     imageFile=fopen("./output_image.ppm","wb");
63     if(imageFile==NULL){
64         perror("ERROR: Cannot open output file");
65         exit(EXIT_FAILURE);
66     }
```

```
67
68      // Configure the file
69      fprintf(imageFile,"P6\n");                // P6 filetype
70      fprintf(imageFile,"%d␣%d\n", M, N);       // dimensions
71      fprintf(imageFile,"255\n");               // Max pixel
72
73      // Write the image
74      fwrite(image_array, 1, M*N*C, imageFile);
75
76      // Close the file
77      fclose(imageFile);
78  }
79
80  __global__ void invertKernel(uint8_t* input, uint8_t* output
        , int size) {
81      int tid = blockIdx.x * blockDim.x + threadIdx.x;
82      int stride = blockDim.x * gridDim.x;
83
84      for (int i = tid; i < size; i += stride) {
85          output[i] = 255 - input[i];
86      }
87  }
88
89  void run_test(uint8_t* d_in, uint8_t* d_out, int size, int
        blocks, int threads) {
90      cudaEvent_t start, stop;
91      cudaEventCreate(&start);
92      cudaEventCreate(&stop);
93      const int iterations = 10;
94      float total_time = 0.0f;
95
96      invertKernel<<<1, 256>>>(d_in, d_out, size);
97      cudaDeviceSynchronize();
98
99      for (int i = 0; i < iterations; i++) {
100         cudaEventRecord(start);
101         invertKernel<<<blocks, threads>>>(d_in, d_out, size)
                ;
102         cudaEventRecord(stop);
103         cudaEventSynchronize(stop);
104         float ms = 0;
105         cudaEventElapsedTime(&ms, start, stop);
106         total_time += ms;
107     }
108     float avg_time = total_time / iterations;
109     printf("%-10d␣|␣%-10d␣|␣%-15.5f\n",
110            blocks, threads,  avg_time);
111     cudaEventDestroy(start);
112     cudaEventDestroy(stop);
113 }
```

```
114
115 // --- MAIN ---
116
117 int main(void) {
118     uint8_t* h_image_in = get_image_array();
119     int size = M * N * C;
120     uint8_t* h_image_out = (uint8_t*)malloc(size);
121     uint8_t *d_in, *d_out;
122
123     cudaMalloc((void**)&d_in, size);
124     cudaMalloc((void**)&d_out, size);
125     cudaMemcpy(d_in, h_image_in, size,
        cudaMemcpyHostToDevice);
126
127     printf("\n%-10s | %-10s | %-15s\n",
128            "Blocks", "Tot. Threads", "Avg Time (ms)");
129     printf("
        ------------------------------------------------------------\
        n");
130     run_test(d_in, d_out, size, 3, 512);
131
132     int thread_tests[] = {5, 16, 31, 32, 33, 80, 96, 128,
        250, 256, 512, 1024};
133
134     for(int j = 1; j < 100; j = j + 5){
135         for(int i = 0; i < 12; i++) {
136             run_test(d_in, d_out, size, j, thread_tests[i]);
137         }
138     }
139     printf("
        ------------------------------------------------------------\
        n");
140
141
142     // Resultaat ophalen voor validatie
143     cudaMemcpy(h_image_out, d_out, size,
        cudaMemcpyDeviceToHost);
144     save_image_array(h_image_out);
145     printf("\nAfbeelding opgeslagen als output_image.ppm\n")
        ;
146
147     // Cleanup
148     cudaFree(d_in);
149     cudaFree(d_out);
150     free(h_image_out);
151     free(h_image_in - OFFSET); // Vergeet de offset niet bij
        free!
152
153     return 0;
154 }
```

4.2 Code voor oefening 2, deel 1

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run 'convert <name>.<extension> <name>.ppm
 *
 */

 // Thread divergence deel van de Warp kunnen complexere berekeningen doen waardoor
 // de eenvoudigere threads in een Warp moeten wachten tot de laatste thread klaar is.

#include <cstdint>      // Data types
#include <iostream>     // File operations

// #define M 512        // Lenna width
// #define N 512        // Lenna height
#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


uint8_t* get_image_array(void){
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M}, ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile=fopen("./anna.ppm","rb");
    if(imageFile==NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
```

```
41      }
42
43      // Initialize empty image array
44      uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(
            uint8_t)+OFFSET);
45
46      // Read the image
47      fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t
            )+OFFSET, imageFile);
48
49      // Close the file
50      fclose(imageFile);
51
52      // Move the starting pointer and return the flattened
            image array
53      return image_array + OFFSET;
54  }
55
56
57  void save_image_array(uint8_t* image_array){
58      /*
59       * Save the data of an (RGB) image as a pixel map.
60       *
61       * Parameters:
62       *  - param1: The data of an (RGB) image as a 1D array
63       *
64       */
65      // Try opening the file
66      FILE *imageFile;
67      imageFile=fopen("./output_image.ppm","wb");
68      if(imageFile==NULL){
69          perror("ERROR: Cannot open output file");
70          exit(EXIT_FAILURE);
71      }
72
73      // Configure the file
74      fprintf(imageFile,"P6\n");               // P6 filetype
75      fprintf(imageFile,"%d %d\n", M, N);      // dimensions
76      fprintf(imageFile,"255\n");              // Max pixel
77
78      // Write the image
79      fwrite(image_array, 1, M*N*C, imageFile);
80
81      // Close the file
82      fclose(imageFile);
83  }
84
85
86  __global__ void invertKernel(uint8_t* input, uint8_t* output
        , int size) {           // data array zit de code als
```

```
    volgt in:  RGBRGB
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = tid; i < size; i += stride) {
        if((i % 3) == 0){
            if (i >= 6 && i < size - 6) {       // blur
                maken op basis van aangrenzende rode pixels
                output[i] = (uint8_t)(
                    0.1f * input[i-6] +
                    0.25f * input[i-3] +
                    0.5f * input[i] +
                    0.25f * input[i+3] +
                    0.1f * input[i+6]
                );
            }
        }
        else {output[i] = 255 - input[i];}
    }
}


void run_test(uint8_t* d_in, uint8_t* d_out, int size, int
    blocks, int threads) {
    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);

    const int iterations = 10;
    float total_time = 0.0f;

    invertKernel<<<1, 256>>>(d_in, d_out, size);
        //verkeerde tijdsmeting voorkomen
    cudaDeviceSynchronize();

    for (int i = 0; i < iterations; i++) {
        // 10 keer code runnen om een gemiddelde tijd te
        bekomen
        cudaEventRecord(start);

        invertKernel<<<blocks, threads>>>(d_in, d_out, size)
            ;

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);

        float ms = 0;
        cudaEventElapsedTime(&ms, start, stop);
        total_time += ms;
    }
```

```
130
131     float avg_time = total_time / iterations;
132     // Bereken het aantal warps dat de hardware moet
            inplannen
133     // Formule: blocks * ceil(threads / 32)
134
135
136     printf("%-10d | %-10d | %-15.5f\n",
137            blocks, threads, avg_time);
138
139     cudaEventDestroy(start);
140     cudaEventDestroy(stop);
141 }
142
143 // --- MAIN ---
144
145 int main(void) {
146     uint8_t* h_image_in = get_image_array();
147     int size = M * N * C;
148     uint8_t* h_image_out = (uint8_t*)malloc(size);
149     uint8_t *d_in, *d_out;
150
151     cudaMalloc((void**)&d_in, size);
152     cudaMalloc((void**)&d_out, size);
153
154     cudaMemcpy(d_in, h_image_in, size,
            cudaMemcpyHostToDevice);
155
156     printf("\n%-10s | %-10s | %-15s\n",
157            "Blocks", "Threads/Block", "Avg Time (ms)");
158     printf("
            ------------------------------------------------------------------\
            n");
159
160     run_test(d_in, d_out, size, 1, 16);
161
162     int thread_tests[] = {5, 16, 31, 32, 33, 80, 96, 128,
            250, 256, 512, 1024};        // threads aanpassen om
            volledige of gedeeltelijke Warps te bekomen
163
164     for(int j = 1; j < 100; j = j + 5){
165         for(int i = 0; i < 12; i++) {
166             run_test(d_in, d_out, size, j, thread_tests[i]);
167         }
168     }
169
170     cudaMemcpy(h_image_out, d_out, size,
            cudaMemcpyDeviceToHost);
171     save_image_array(h_image_out);
172
```

```
    cudaFree(d_in);
    cudaFree(d_out);
    free(h_image_out);
    free(h_image_in - OFFSET);
    return 0;
}
```

4.3 Code voor oefening 2, deel 2

```
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <cuda_runtime.h>

// Definities voor de VR afbeelding (Pas aan indien nodig)
#define M 960         // Breedte
#define N 1280        // Hoogte
#define C 3           // Kleurkanalen (RGB)
#define OFFSET 16     // Header lengte van PPM

// --- IMAGE HELPERS ---

uint8_t* get_image_array(void){
    FILE *imageFile = fopen("./anna.ppm","rb");
    if(imageFile == NULL){
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }
    uint8_t* image_array = (uint8_t*)malloc(M*N*C*sizeof(
        uint8_t)+OFFSET);
    fread(image_array, sizeof(uint8_t), M*N*C*sizeof(uint8_t
        )+OFFSET, imageFile);
    fclose(imageFile);
    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array){
    FILE *imageFile = fopen("./output_image.ppm","wb");
    if(imageFile == NULL){
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }
    fprintf(imageFile,"P6\n%d %d\n255\n", M, N);
    fwrite(image_array, 1, M*N*C, imageFile);
    fclose(imageFile);
}

// --- CUDA KERNEL (OPTIMIZED FOR RRR...GGG...BBB) ---

```

```
38 __global__ void invertKernelOptimized(uint8_t* input,
       uint8_t* output, int pixels) {
39     int tid = blockIdx.x * blockDim.x + threadIdx.x;
40     int stride = blockDim.x * gridDim.x;
41
42     for (int i = tid; i < pixels * 3; i += stride) {    //
           voor rode pixels complexere formule toepassen
43         if (i < pixels) {
44             if (i >= 2 && i < pixels - 2) {
45                 output[i] = (uint8_t)( 0.1f * input[i-2] +
                       0.25f * input[i-1] + 0.5f * input[i]
46                        + 0.25f * input[i+1] + 0.1f * input[i
                           +2]);
47             }
48         }
49         else {  // voor G en B formule uit vorig deel
               toepassen
50             output[i] = 255 - input[i];
51         }
52     }
53 }
54
55
56 void run_test(uint8_t* d_in, uint8_t* d_out, int pixels, int
       blocks, int threads) {
57     cudaEvent_t start, stop;
58     cudaEventCreate(&start);
59     cudaEventCreate(&stop);
60     cudaDeviceSynchronize();
61
62     const int iterations = 10;
63     float total_time = 0.0f;
64
65     for (int i = 0; i < iterations; i++) {
66         cudaEventRecord(start);
67         invertKernelOptimized<<<blocks, threads>>>(d_in,
               d_out, pixels);
68         cudaEventRecord(stop);
69         cudaEventSynchronize(stop);
70
71         float ms = 0;
72         cudaEventElapsedTime(&ms, start, stop);
73         total_time += ms;
74     }
75
76     printf("%-10d␣|␣%-10d␣|␣%-15.5f\n", blocks, threads,
           total_time / iterations);
77
78     cudaEventDestroy(start);
79     cudaEventDestroy(stop);
```

```
80  }
81
82  // --- MAIN ---
83
84  int main(void) {
85      uint8_t* h_image_raw = get_image_array();
86      int pixels = M * N;
87      int size = pixels * 3;
88      uint8_t* h_reformatted = (uint8_t*)malloc(size);
89      uint8_t* h_out_reformatted = (uint8_t*)malloc(size);
90      uint8_t* h_final_output = (uint8_t*)malloc(size);
91      uint8_t *d_in, *d_out;
92      int thread_tests[] = {5, 16, 31, 32, 33, 80, 96, 128,
            250, 256, 512, 1024};
93
94
95      for (int i = 0; i < pixels; i++) {
                                    // data omzetten naar RRR
            ... GGG ... BBB formaat
96          h_reformatted[i] = h_image_raw[i * 3];           //
                R
97          h_reformatted[pixels + i] = h_image_raw[i * 3 + 1];
                // G
98          h_reformatted[2 * pixels + i] = h_image_raw[i * 3 +
                2]; // B
99      }
100
101     cudaMalloc((void**)&d_in, size);
102     cudaMalloc((void**)&d_out, size);
103     cudaMemcpy(d_in, h_reformatted, size,
            cudaMemcpyHostToDevice);
104     run_test(d_in, d_out, pixels, 1, thread_tests[0]);
                    // dummy test
105
106     printf("\n%-10s␣|␣%-10s␣|␣%-15s\n", "Blocks", "Threads/
            Bl", "Avg␣Time␣(ms)");
107     printf("
            ------------------------------------------------------------\
            n");
108     for(int j = 1; j < 100; j = j + 5){
109         for(int i = 0; i < 12; i++) {
110             run_test(d_in, d_out, pixels, j, thread_tests[i
                    ]);
111         }
112     }
113
114     cudaMemcpy(h_out_reformatted, d_out, size,
            cudaMemcpyDeviceToHost);
115
116     for (int i = 0; i < pixels; i++) {
```

```
117         h_final_output[i * 3]     = h_out_reformatted[i];
                                // R
118         h_final_output[i * 3 + 1] = h_out_reformatted[pixels
                + i];     // G
119         h_final_output[i * 3 + 2] = h_out_reformatted[2 *
                pixels + i]; // B
120     }
121
122     save_image_array(h_final_output);
123     cudaFree(d_in); cudaFree(d_out);
124     free(h_reformatted); free(h_out_reformatted); free(
            h_final_output);
125     free(h_image_raw - OFFSET);
126
127     return 0;
128 }
```